

BSc (Hons) Computer Science with Software Engineering
Coventry University - Delivered by NIBM

NB6007CEM - Web API Development

Composite Resources and Representations



Day 3 (AM) - Session 5 - VehicleComposite

Niranga Dharmaratna - niranga@nibm.lk

From Provisional to Authoritative



In Session 4, every JSON shape in your API was provisional - generated by the model, not designed by you.

Today that ends.

By the close of this session, every field name, every nested object, and every array in your API responses is a design decision you can justify against REST API Design Guidelines - §6.

Session 6 this afternoon is when you implement those decisions.

After S5 + S6: your deployed API returns authoritative, binding JSON shapes on every route - an API design dimension deliverable.

Session 5 - Representations and Composite Resources

Day 3 AM - closing the resource model, choosing the representation

- Distinguish resource from representation; justify the application/json decision
- Design the vehicle composite resource embedding the most recent location ping
- Design the last-position processing function resource as a derived computation
- Establish authoritative JSON shapes for all routes built in S3-S4

REST API Design Pipeline

1. Data Model - done

2. Derive Resources - done

3. Representations - TODAY

4. URIs (S3-S4) - done

5. HTTP Methods

6. Special Behaviour

7. Errors

8. Security

9. Maturity

A - Resources and Their Representations

REST API Design Guidelines - §6

Resources and Representations - §6



- A resource is abstract - the Province, the Vehicle, the LocationPing.
- A representation is one rendering of that resource's information in a specific format - JSON, XML, plain text.
- The same Vehicle can render as `application/json` or `application/xml` - the `vehicle_id` and `reg_number` do not change; only the rendering does.

Choosing a representation means choosing a media type and a data structure for every resource in the API.

ANALOGY

A police report in Sinhala or English: the same case number, time, and offence section, rendered in a different language. Limit: a report language is fixed at issue; an HTTP server negotiates format on each request.



REPRESENTATION - §6

**The resource stays the same.
The URI stays the same.
Only the format changes.**

Two routes for the same vehicle - one returning JSON, one returning XML - is not a representation decision. It is a resource model error.

Generator Mistakes - Representations

MISTAKE: Two Routes for One Resource

Generator creates:

GET /vehicles - returns JSON

GET /vehicles.xml - returns XML

These are not two representations - they are two separate resources at different URIs.

§5.1: one resource, one URI. The format lives in the Accept request header, not in the path.

Fix: one route, one URI; format is negotiated via HTTP headers.

MISTAKE: Wrong Content-Type Header

Generator uses `res.send()` instead of `res.json()`

Result: Content-Type: text/plain in the response.

Every JSON client silently receives malformed data - no error is thrown at the server.

Fix: `res.json()` on every route. Verify Content-Type in Postman before S6 ends.

The Representation Decision - application/json



- **Decision for this API: application/json for all responses.**
- **Native to Node.js and JavaScript - no parser overhead, direct object handling.**
- **Simpler structure for the tuk-tuk domain - no attribute vs element ambiguity as in XML.**

Dominant in modern REST API practice - every client library handles it natively.

Content Negotiation - awareness only (§10.1)

Client sends Accept: application/json;q=0.9, application/xml;q=0.6 - server returns best match, or 406 Not Acceptable.

This API supports one media type. Negotiation is trivial - the Accept header still appears at viva.

B - Composite Resources

REST API Design Guidelines - §4.3

Composite Resources - §4.3



- Groups of different entity types typically retrieved or deleted as a whole become composite resources.
- A composite bundles entities whose information is only useful together - retrieving the parts separately wastes round trips.
- In the tuk-tuk system: a police dispatcher viewing a vehicle always needs both the vehicle attributes and its last known position - two separate calls for every vehicle on the map is poor design.

Design decision: GET /vehicles/{vehicle-id} returns the vehicle composite - vehicle attributes plus the most recent LocationPing, embedded.

ANALOGY

A tuk-tuk registration certificate bundles vehicle details, owner, and fitness test result - never retrieved separately. The composite resource works the same way: one request, one complete picture.

Limit: a certificate is static; this composite returns live data updated on every new ping.

GET /vehicles/{vehicle-id} - The Vehicle Composite

```
{
  "vehicle_id": "WP-1234",
  "reg_number": "WP CAA-1234",
  "device_id": "DEV-001",
  "station_id": "ST-05",
  "last_ping": {
    "ping_id": "PNG-98765",
    "timestamp": "2025-01-15T10:30:00Z",
    "lat": 6.9271,
    "lng": 79.8612,
    "speed": 25
  }
}
```

Vehicle attributes

vehicle_id, reg_number, device_id, station_id - your data model decisions from S1.

last_ping - embedded LocationPing

The most recent ping only. Full history lives at /vehicles/{id}/pings - taught with pagination in S9.

We use GET for this route - Session 7 explains exactly why.

Generator Mistakes - Composite Resources

MISTAKE: Flat Location Fields

Generator adds: `last_lat`, `last_lng`, `last_speed` as flat fields on the vehicle object.

This is the S1 data model mistake resurfacing at the representation level - collapsing the LocationPing entity back into three fields.

What is lost: the ping's timestamp, `ping_id`, and its identity as a time-series record.

Correct: embed the full `last_ping` object, not three flat coordinate fields.

MISTAKE: Embedding the Full History

Generator embeds "pings": [...] - the entire history array inside the vehicle composite.

The full history does not belong in the composite. It belongs at GET `/vehicles/{id}/pings`, with pagination (S9).

A composite shows the most recent state - one embedded ping, not a growing array.

Sending 500 pings per vehicle in every GET `/vehicles/{id}` call is a performance and design failure.

C - Processing Function Resources

REST API Design Guidelines - §4.5

Processing Function Resources - §4.5



- Processing function resources provide access to derived computations - results not directly stored in the data model.
- In the tuk-tuk system: a police officer sometimes needs only the last known position of a specific vehicle - not the full composite. This is computed from LocationPing (find the most recent ping for this vehicle_id), not stored as a separate field.

Naming (§5.1): processing function resources that represent derived reads are named as nouns. Verbs are only correct for action resources (controller type).

ANALOGY

The tuk-tuk stand supervisor derives "Where is WP-1234?" from the last radio log - no pre-stored current-location field. The /last-position resource is that same computation. Limit: a supervisor might misremember; the API always returns the exact most recent ping.

GET /vehicles/{vehicle-id}/last-position

```
{
  "vehicle_id": "WP-1234",
  "timestamp": "2025-01-15T10:30:00Z",
  "lat": 6.9271,
  "lng": 79.8612,
  "speed": 25
}
```

URI segment: last-position (a noun, not a verb)

This is a scoped processing function: it lives under /vehicles/{id} because it is derived from that vehicle's pings only.

We use GET for this route - Session 7 explains exactly why.

Position only - no vehicle metadata.
Call /vehicles/{id} when you also need vehicle attributes.

Call /last-position for position-only dashboard updates - lighter response, faster render for a map overlay that needs coordinates only.

Coursework connection:

New route added in S6. API design dimension. §4.5. Document the design decision in your report.

Generator Mistakes - Processing Function Resources

MISTAKE 1: Verb in the URI Path

Generator names the route: GET /getLastKnownPosition or GET /vehicles/:id/calculatePosition

Both violate §5.1: verbs in paths. Processing function resources that are derived reads are named as nouns.

Correct path: GET /vehicles/{vehicle-id}/last-position

MISTAKE 2: Duplication - Field AND Separate Resource

Generator makes last-position a flat field on the vehicle composite AND a separate route.

The design decision is explicit: composite embeds last_ping for when vehicle context is needed; /last-position is a lightweight processing function for position-only reads.

One or the other per access pattern - not both. Document the decision in your report.

Controller Resources - §4.4 (For Completeness)

What are Controller Resources?

Used when multiple resources must be updated atomically to maintain data consistency.

Named as verbs (§5.1) - the only resource type where a verb URI segment is correct.

Example (not built): POST /transfer-vehicle would update a Vehicle's station_id and create an audit record in one atomic call, with no partial-update risk.

Why This API Has None

IoT tracking devices only write location pings - a single-resource write each time.

Police users only read - no multi-resource atomic updates arise in the core scope.

No controller resources are needed or built in this module. Knowing the type exists helps you justify their absence in your report.

Resource types now complete: Atomic (S1) - Collection (S3) - Composite (today) - Processing Function (today) - Controller (today, for completeness).

E - All Shapes Are Now Binding

Every field name on the following slides is a justified design decision from this point forward.

Authoritative Shapes - Atomic Resources

Province

```
{
  "province_id": "PV-01",
  "name": "Western Province"
}
```

District

```
{
  "district_id": "DT-03",
  "name": "Colombo",
  "province_id": "PV-01"
}
```

PoliceStation

```
{
  "station_id": "ST-05",
  "name": "Colombo Fort",
  "district_id": "DT-03"
}
```

Collection shape: an array of the corresponding atomic objects.

GET /provinces returns [{ "province_id": "PV-01", "name": "Western Province" }, ...]

No metadata fields yet (count / next / previous) - those belong to S9 pagination.

Authoritative Shapes - Composite and Processing Function

Vehicle Composite

```
{
  "vehicle_id": "WP-1234",
  "reg_number": "WP CAA-1234",
  "device_id": "DEV-001",
  "station_id": "ST-05",
  "last_ping": {
    "ping_id": "PNG-98765",
    "timestamp": "2025-01-15T10:30:00Z",
    "lat": 6.9271,
    "lng": 79.8612,
    "speed": 25
  }
}
```

LocationPing

```
{
  "ping_id": "PNG-98765",
  "vehicle_id": "WP-1234",
  "timestamp": "2025-01-15T10:30:00Z",
  "lat": 6.9271,
  "lng": 79.8612,
  "speed": 25
}
```

last-position (processing function)

```
{
  "vehicle_id": "WP-1234",
  "timestamp": "2025-01-15T10:30:00Z",
  "lat": 6.9271,
  "lng": 79.8612,
  "speed": 25
}
```

Design Decisions - Field Naming and Response Shape

Field Naming - snake_case Across All Shapes

Decision: snake_case for every field in every resource.

Consistent with seed.json structure committed in S2.

Generator mistake: camelCase fields (vehicleId, regNumber, stationId) appearing in some endpoints - violates the consistency requirement.

Document this decision in your report architecture section. Every route must use the same convention.

Avoid - The Envelope Wrapper

Generator wraps responses: { "data": [...], "status": "success" }

§6 does not specify this wrapper - it adds complexity without value at this stage.

Correct shapes: flat arrays and flat objects exactly as defined on slides 19-20.

A wrapper may be introduced in S11 if the error schema requires it. Until then, remove it.

Representation Demo

Open the demo file:

NB6007CEM_S5_VehicleComposite_Demo_RepAndURI.html

- Left panel: select a resource, toggle JSON / XML.
- Watch the URI hold still while the body changes.
- Right panel: type a vehicle ID.

Watch /vehicles/{id}/pings assemble in real time.

Ask the room:

"When you flip from JSON to XML - what stays the same?"

"When you type a different vehicle ID - what changes in the URI?"

Both questions target the resource / representation / template distinction from slides 5-6.

Review Your S4 Routes

- Open your `/vehicles/:vehicleId` route from S4.
- Does it return a composite (vehicle attributes + embedded `last_ping`) or an atomic (vehicle attributes only)?
- Is `last_ping` nested as an object, or are the fields flat (`last_lat`, `last_lng`)?
- What Content-Type header does your API send? Check in Postman or curl.

List every change needed before the end of S6 this afternoon.

Questions & Discussion

*S6 this afternoon: implement the authoritative shapes,
add /last-position, and redeploy.*

Disclaimer and Copyrights

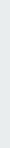


The information presented in this presentation is intended for general informational purposes only. It is not meant to be comprehensive or to constitute professional advice. Any reliance you place on such information is strictly at your own risk. While I strive to keep the information accurate and up-to-date, I make no representations or warranties of any kind, express or implied, about the completeness, accuracy, reliability, suitability, or availability concerning the content within this presentation.

This presentation may include references or links to third-party websites or content. I do not endorse the views expressed or the information provided on such external sites. I am not responsible for the content of any linked site or any link contained in a linked site.

Any action you take upon the information presented in this presentation is strictly at your own discretion. I will not be liable for any losses or damages in connection with the use of this information.

This disclaimer is subject to change without notice.



@ Images and artworks used in this presentation are used for general informational purposes only. It may contain copyrighted material, the use of which may not have been specifically authorised by the copyright owner. This material is available in an effort to explain issues relevant to the topic. This should constitute a 'fair use' of any such copyrighted material.

Last modified: 20 Jun 2026